Pearls AQI Predictor

Final Project Report

Mansoor Abid

August 2026

Contents

Executive Summary **2**
- End-to-End AQI Prediction System . . . 2
- Scalable, Automated Pipeline . . . 2
- Interactive Dashboard . . . 2
- Project Documentation . . . 3

System Architecture **4**
- Technology Stack . . . 4

Feature Engineering & Data Pipeline **4**

Model Training & Evaluation **5**
- Training Strategy . . . 5
- Model Promotion . . . 6

Explainability (SHAP) **6**

Challenges & Solutions **6**

Future Work **7**

Program: 10Pearls Internship

Executive Summary

This report documents the final deliverable for the 10Pearls internship project: a production-ready MLOps system for air quality forecasting in Lahore, Pakistan.

Key Results

- **48% skill score** over naive persistence baseline on the 1-day horizon
- **9.26 RMSE** (AQI points) for next-day forecasts; down from ∼18 in initial experiments
- **Fully automated** three-pipeline architecture (feature, training, deploy)
- **5 model architectures** evaluated daily; Random Forest consistently promoted to production

End-to-End AQI Prediction System

The system ingests hourly weather and pollutant data from the Open-Meteo API, engineers 30+ predictive features, and trains five model architectures (Ridge Regression, Random Forest, XGBoost, Feed-Forward NN, LSTM). It forecasts the European AQI for Lahore up to 72 hours ahead.

Scalable, Automated Pipeline

A serverless MLOps architecture using GitHub Actions and Hopsworks Cloud:

- **Feature Pipeline:** Runs 4× daily, pushing fresh data to the Hopsworks Feature Store
- **Training Pipeline:** Runs daily, retraining all models, promoting the best performer (lowest RMSE) to the Model Registry, and generating SHAP explainability plots
- **Deploy Pipeline:** Triggered on push to GitHub

Interactive Dashboard

A real-time Streamlit dashboard loads production models directly from the Hopsworks registry. Features include:

- Real-time AQI gauge with WHO threshold color-coding
- Pollutant concentration cards (PM2.5, PM10, NO_2, O_3, etc.)
- 7-day historical trend chart
- 3-day forecast with confidence intervals
- Dynamic SHAP feature importance visualizations

Project Documentation

This report details the system architecture, feature engineering methodology, model evaluations, pipeline automation, and operational lessons learned.

System Architecture

The system follows a standard enterprise MLOps flow:

`Data Source -> Feature Pipeline -> Feature Store -> Training Pipeline -> Model Registry -> Inference Layer -> Dashboard/API`

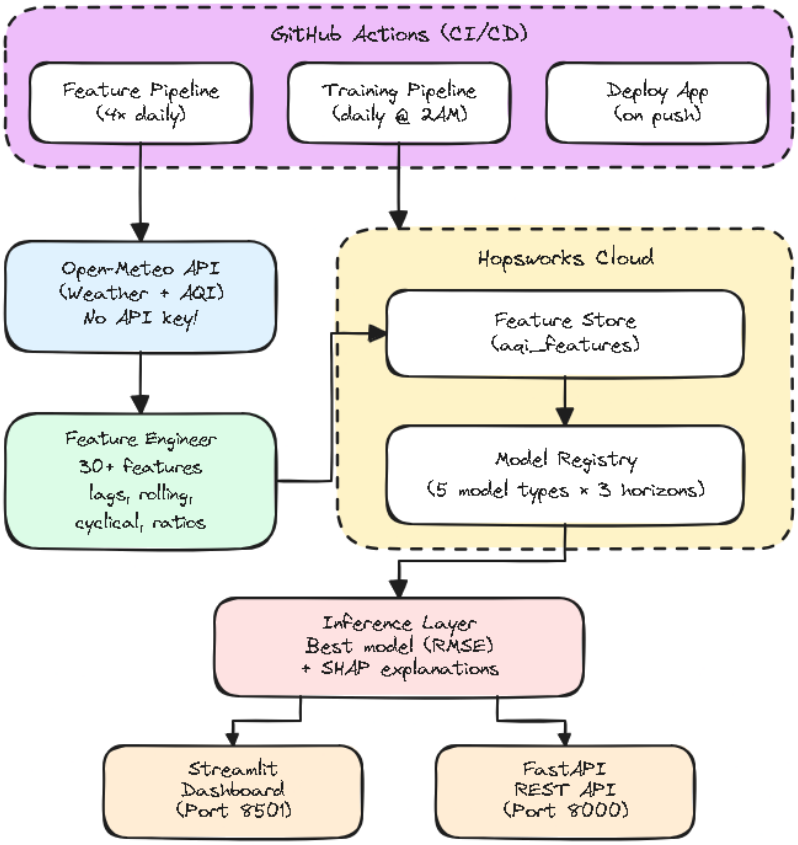

Figure 1: End-to-end system architecture showing data flow from ingestion through to the Streamlit dashboard and FastAPI endpoint.

Technology Stack

Feature Engineering & Data Pipeline

The `AQIFeatureEngineer` transformer produces 30+ features from raw hourly data:

- **Temporal & Cyclical:** Hour, day of week, and month encoded via sine/cosine transformations to capture diurnal and seasonal cycles
- **Wind Vectors:** Wind speed and direction decomposed into u and v components
- **Pollutant Ratios:** PM2.5/PM10 ratio and NO_2/O_3 interaction to capture photochemical activity

Component	Technology
Data Ingestion	Open-Meteo API (free, no API key)
Feature Store / Model Registry	Hopsworks Cloud (managed Kafka ingestion)
ML Frameworks	scikit-learn, XGBoost, PyTorch
Explainability	SHAP (SHapley Additive exPlanations)
Dashboard	Streamlit + Plotly
REST API	FastAPI + Uvicorn
CI/CD	GitHub Actions
Package Management	uv (Astral)
Data Validation	Evidently AI

Table 1: Technology stack

- **Climate Trends:** 7-day temperature trends and 7-day rain accumulation
- **Lag Features & Rolling Statistics:** AQI at 1h, 3h, 6h, 24h lags; rolling means over 4h, 6h, 12h, 24h windows; exponential weighted moving average (6h)

Data is validated for schema integrity and drift using Evidently AI before being written to the Hopsworks Feature Store.

Model Training & Evaluation

Five architectures are evaluated for each forecast horizon (1-day, 2-day, 3-day):

1. **Ridge Regression:** Linear baseline with RobustScaler preprocessing
2. **Random Forest:** 300 trees, max depth 12
3. **XGBoost:** Gradient boosting with early stopping (learning rate 0.03)
4. **Feed-Forward NN (PyTorch):** 128 $\rightarrow$ Dropout $\rightarrow$ 64
5. **LSTM (PyTorch):** Sequence model with 24-hour lookback

Training Strategy

Rather than predicting absolute AQI directly, models predict the **AQI delta** (change from current AQI):

$$\text{predicted\_AQI} = \text{current\_AQI} + \text{predicted\_delta} \tag{1}$$

This anchors short-term predictions to the observed state, significantly improving accuracy over direct prediction.

Model Promotion

Models are ranked by **Skill Score** which is improvement over a naive persistence forecast (assuming AQI remains unchanged):

$$\text{Skill Score} = 1 - \frac{\text{RMSE}_{\text{model}}}{\text{RMSE}_{\text{naive}}} \tag{2}$$

The best model's artifacts are serialized and uploaded to the Hopsworks Model Registry. Random Forest has proven the most robust architecture across all horizons for this dataset.

Horizon	Model	Version	RMSE
Day 1 (24h)	random_forest	7	9.26
Day 2 (48h)	random_forest	3	9.49
Day 3 (72h)	random_forest	3	10.17

Table 2: Current production models active in the Model Registry (as of 13 August 2026)

Performance Trajectory: RMSE has improved steadily from ∼18 in early Colab experiments to the current 9.26 as training data accumulated and the automated retraining pipeline matured.

Explainability (SHAP)

SHAP values are computed for every promoted model to ensure predictions are interpretable and auditable. Plots are bundled with model artifacts and displayed on the dashboard.

Key Findings (Current Production Model)

1. `month_cos`: Seasonal cycle is the strongest predictor, reflecting monsoon and winter pollution patterns
2. `aqi_lag_6h`: Recent pollution levels effectively capture short-term persistence
3. `wind_speed_10m`: Wind speed dictates how quickly pollutants disperse from the city center
4. `pm10`, `pm2_5`: Direct particulate measurements remain critical baseline signals

Feature rankings evolve as the model retrains on fresh data; the above reflects the current production version.

Challenges & Solutions

1. **Hopsworks API Key Scopes:** Kafka-based feature ingestion failed due to missing `KAFKA` permission scopes. Resolved by updating token permissions in the Hopsworks

console.

2. **Scaler Serialization:** Early inference failed because the fitted RobustScaler was not persisted alongside model weights. Fixed by updating the `save()` method in `sklearn_models.py` to bundle both via `joblib`.
3. **CI/CD Artifact Handling:** GitHub Actions initially failed when committing SHAP images back to the repository. Resolved architecturally by saving SHAP plots directly into the Hopsworks Model Registry artifacts directory, eliminating the need for git-based artifact persistence.

Future Work

- **Multi-city support:** Extend the pipeline to Karachi and Islamabad
- **Ensemble predictions:** Combine the top 3 models via weighted averaging for improved robustness
- **Drift-triggered retraining:** Replace cron-based retraining with trigger-based retraining when data drift is detected